Navegación entre pantallas

Curso 2026 - 2027



Contexto

Una carta de un restaurante que:

- Puedas ver la lista de platos
- Al seleccionar un plato, te muestre una foto y una descripción

Luego hablaremos de la **UI adaptativa**:

- Al principio aparecerá el mismo comportamiento entre un *smartphone* y en escritorio/web, pero...
- Pensad en el *layout* de Spotify o en WhatsApp:
  - En smartphone, te cambia entre pantallas al seleccionar y mostrar una canción / lista / chat.
  - En escritorio/web, tienes todo en una sola pantalla.



NAVEGACIÓN

Librería `navigation`

Principales conceptos de la librería `navigation` para navegar entre diferentes pantallas:

- **Grafo de navegación**: describe todos los destinos posibles y sus conexiones entre ellos.
- **Destino**: nodo del grafo de navegación al que se puede navegar.
- **Ruta**: define un destino y los argumentos necesarios para navegar, sin especificar nada de la UI.

Librería `navigation`

1. Define las rutas.
2. Crea un `NavController` que proporciona toda la funcionalidad para navegar entre destinos.
3. Crea un composable `NavHost` que contiene el grafo de navegación.

```kotlin
// Route.kt en el mismo directorio de App.kt
@Serializable
sealed interface Route {

    @Serializable
    data object Carta : Route

    @Serializable
    data class DetallePlato(val platoId: Int) : Route
}
```

`@Serializable` — Proceso de convertir datos usados por una aplicación en un formato que puede ser transferido a través de una red, o almacenados en una base de datos o en archivos.

`data object` — Usado para acciones SIN datos asociados.

`data class` — Usado para acciones CON datos asociados.

```kotlin
@Serializable
sealed interface Route

@Serializable
data object Carta : Route

@Serializable
data class DetallePlato(val platoId: Int) : Route
```

`@Serializable`	Proceso de convertir datos usados por una aplicación en un formato que puede ser transferido a través de una red, o almacenados en una base de datos o en archivos.
`data object`	Usado para acciones SIN datos asociados.
`data class`	Usado para acciones CON datos asociados.

- Crea un `NavController` que proporciona toda la funcionalidad para navegar entre destinos.
- Crea un composable `NavHost` que contiene el grafo de navegación.

```
@Composable
fun App(appContainer: AppContainer) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Route.Carta) {
        composable<Route.Carta> { /* ... */ }
        composable<Route.DetallePlato> { /* ... */ }
    }
}
```

Navegar entre destinos

```
Button(onClick = { navController.navigate(DetallePlato) }) {
    Text("Ver la descripción del plato.")
}
```

Navegar entre destinos

```
Button(onClick = { navController.navigate(DetallePlato) }) {
    Text("Ver la descripción del plato.")
}
```

```
@Serializable
data class DetallePlato(val platoId: Int) : Route
```

Navegar entre destinos

```
Button(onClick = { navController.navigate(DetallePlato) }) {
    Text("Ver la descripción del plato.")
}
```

```
@Serializable
data class DetallePlato(val platoId: Int) : Route
```

```
Button(onClick = { navController.navigate(DetallePlato(plato.id)) }) {
    Text("Ver la descripción del plato.")
}
```

Navegar entre destinos

¿Por qué solo pasamos el ID en vez del plato entero?

Al navegar entre destinos, es recomendable pasar la información mínima innecesaria. De esta manera, prevenimos que se pierdan datos durante los cambios de la configuración o al actualizar los objetos referenciados.

Mejor pasamos la ID y luego volvemos a buscar la información en nuestra base de datos.

```
Button(onClick = { navController.navigate(DetallePlato(plato.id)) }) {
    Text("Ver la descripción del plato.")
}
```

Recibir el dato en el destino

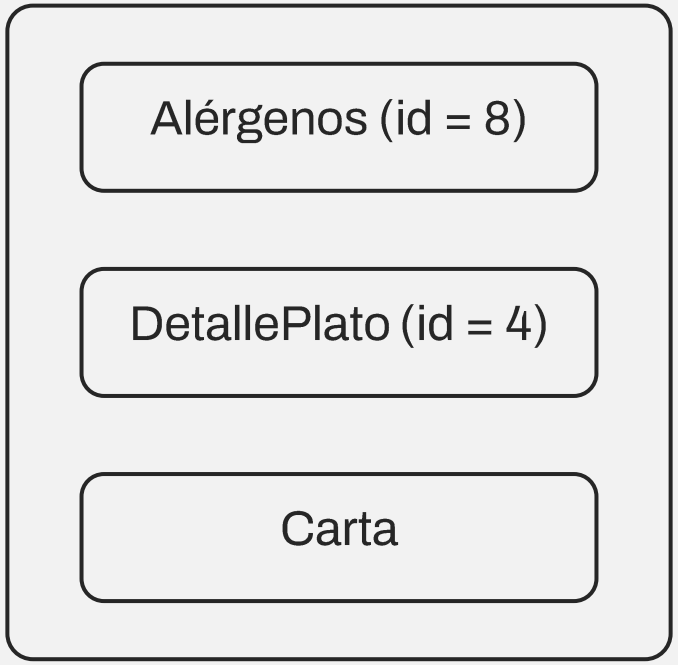
```
composable<Route.DetallePlato> { it ->
        val route: Route.DetallePlato = it.toRoute()
        // route.platoId
}
```

Recibir el dato en el destino

```
composable<Route.DetallePlato> { backStackEntry ->
        val route: Route.DetallePlato = backStackEntry.toRoute()
        // route.platoId
}
```

La pila (*stack*)

Cuando navegas en tu app, *Compose Navigation* mantiene una **pila** (*stack*) con el historial de pantallas por las que has pasado



`startDestination`: Define la primera pantalla que verá el usuario al iniciar la app

```
NavHost(navController = navController, startDestination = Route.Carta) {
    composable<Route.Carta> { /* ... */ }
    composable<Route.DetallePlato> { /* ... */ }
}
```

`navController.navigate(…)`: Añade un nuevo destino arriba de la pila

```
Button(onClick = { navController.navigate(DetallePlato(plato.id)) }) {
    Text("Ver la descripción del plato.")
}
```

`popBackStack()`: Elimina el destino de arriba y vuelve al anterior

```
Button(onClick = { navController.popBackStack() }) {
    Text("Volver a la carta.")
}
```

¿Cómo rellenaríamos esto con los que sabemos hasta ahora?

```
@Composable
fun App(appContainer: AppContainer) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Route.Carta) {
        composable<Route.Carta> { /* ... */ }
        composable<Route.DetallePlato> { /* ... */ }
    }
}
```

```kotlin
@Composable
fun App(appContainer: AppContainer) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Route.Carta) {

        composable<Route.Carta> {
            val viewModel = remember {
                CartaViewModel(appContainer.obtenerPlatosUseCase)
            }
            CartaScreenRoot(
                viewModel = viewModel,
                onPlatoClick = { plato ->
                    navController.navigate(Route.DetallePlato(plato.id))
                }
            )
        }

        composable<Route.DetallePlato> { backStackEntry ->
            val route: Route.DetallePlato = backStackEntry.toRoute()
            val viewModel = remember {
                DetallePlatoViewModel(
                    platoId = route.platoId,
                    obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
                )
            }
            DetallePlatoScreenRoot(
                viewModel = viewModel,
                onVolver = { navController.popBackStack() }
            )
        }
    }
}
```

```
composable<Route.Carta> {
    val viewModel = remember {
        CartaViewModel(appContainer.obtenerPlatosUseCase)
    }
    CartaScreenRoot(
        viewModel = viewModel,
        onPlatoClick = { plato ->
            navController.navigate(Route.DetallePlato(plato.id))
        }
    )
}
```

```
composable<Route.Carta> {
    val viewModel = remember {
        CartaViewModel(appContainer.obtenerPlatosUseCase)
    }
    CartaScreenRoot(
        viewModel = viewModel,
        onPlatoClick = { plato ->
            navController.navigate(Route.DetallePlato(plato.id))
        }
    )
}
```

```
class CartaViewModel(
    private val obtenerPlatos: ObtenerPlatosUseCase
) : ViewModel() {

    private val _state = MutableStateFlow(CartaState())
    val state: StateFlow<CartaState> = _state.asStateFlow()

    init {
        viewModelScope.launch {
            _state.update { it.copy(platos = obtenerPlatos()) }
        }
    }
}
```

```
composable<Route.Carta> {
    val viewModel = remember {
        CartaViewModel(appContainer.obtenerPlatosUseCase)
    }
    CartaScreenRoot(
        viewModel = viewModel,
        onPlatoClick = { plato ->
            navController.navigate(Route.DetallePlato(plato.id))
        }
    )
}
```

```
@Composable
fun PizzaScreenRoot(viewModel: CamareroViewModel) {
    val state by viewModel.state.collectAsStateWithLifecycle()

    PizzaScreen(
        state = state,
        onAction = viewModel::onAction
    )
}
```

```
@Composable
fun CartaScreenRoot(
    viewModel: CartaViewModel,
    onPlatoClick: (Plato) -> Unit
) {
    val state by viewModel.state.collectAsStateWithLifecycle()

    CartaScreen(
        state = state,
        onAction = { action ->
            when (action) {
                is CartaAction.OnPlatoClick -> onPlatoClick(action.plato) // navegación
                else -> viewModel.onAction(action) // resto: al ViewModel
            }
        }
    )
}
```

```kotlin
@Composable
fun App(appContainer: AppContainer) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Route.Carta) {

        composable<Route.Carta> {
            val viewModel = remember {
                CartaViewModel(appContainer.obtenerPlatosUseCase)
            }
            CartaScreenRoot(
                viewModel = viewModel,
                onPlatoClick = { plato ->
                    navController.navigate(Route.DetallePlato(plato.id))
                }
            )
        }

        composable<Route.DetallePlato> { backStackEntry ->
            val route: Route.DetallePlato = backStackEntry.toRoute()
            val viewModel = remember {
                DetallePlatoViewModel(
                    platoId = route.platoId,
                    obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
                )
            }
            DetallePlatoScreenRoot(
                viewModel = viewModel,
                onVolver = { navController.popBackStack() }
            )
        }
    }
}
```

```kotlin
composable<Route.DetallePlato> { backStackEntry ->
    val route: Route.DetallePlato = backStackEntry.toRoute()
    val viewModel = remember {
        DetallePlatoViewModel(
            platoId = route.platoId,
            obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
        )
    }
    DetallePlatoScreenRoot(
        viewModel = viewModel,
        onVolver = { navController.popBackStack() }
    )
}
```

```kotlin
composable<Route.DetallePlato> { backStackEntry ->
    val route: Route.DetallePlato = backStackEntry.toRoute()
    val viewModel = remember {
        DetallePlatoViewModel(
            platoId = route.platoId,
            obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
        )
    }
    DetallePlatoScreenRoot(
        viewModel = viewModel,
        onVolver = { navController.popBac
    )
}
```

```kotlin
class DetallePlatoViewModel(
    private val platoId: Int,
    private val obtenerPlatoPorId: ObtenerPlatoPorIdUseCase
) : ViewModel() {

    private val _state = MutableStateFlow(DetallePlatoState())
    val state: StateFlow<DetallePlatoState> = _state.asStateFlow()

    init {
        cargarPlato()
    }

    private fun cargarPlato() {
        viewModelScope.launch {
            _state.update { it.copy(cargando = true) }
            val plato = obtenerPlatoPorId(platoId)
            _state.update { it.copy(cargando = false, plato = plato) }
        }
    }
}
```

```
composable<Route.DetallePlato> { backStackEntry ->
    val route: Route.DetallePlato = backStackEntry.toRoute()
    val viewModel = remember {
        DetallePlatoViewModel(
            platoId = route.platoId,
            obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
        )
    }
    DetallePlatoScreenRoot(
        viewModel = viewModel,
        onVolver = { navController.p
    )
}
```

```
@Composable
fun DetallePlatoScreenRoot(
    viewModel: DetallePlatoViewModel
    onVolver: () -> Unit
) {
    val state by viewModel.state.collectAsStateWithLifecycle()

    DetallePlatoScreen(
        state = state
        onAction = { action ->
            is DetallePlatoAction.OnVolver -> onVolver()
            else -> viewModel.onAction(action)
        }
    )
}
```

```kotlin
@Composable
fun App(appContainer: AppContainer) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Route.Carta) {

        composable<Route.Carta> {
            val viewModel = remember {
                CartaViewModel(appContainer.obtenerPlatosUseCase)
            }
            CartaScreenRoot(
                viewModel = viewModel,
                onPlatoClick = { plato ->
                    navController.navigate(Route.DetallePlato(plato.id))
                }
            )
        }

        composable<Route.DetallePlato> { backStackEntry ->
            val route: Route.DetallePlato = backStackEntry.toRoute()
            val viewModel = remember {
                DetallePlatoViewModel(
                    platoId = route.platoId,
                    obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
                )
            }
            DetallePlatoScreenRoot(
                viewModel = viewModel,
                onVolver = { navController.popBackStack() }
            )
        }
    }
}
```

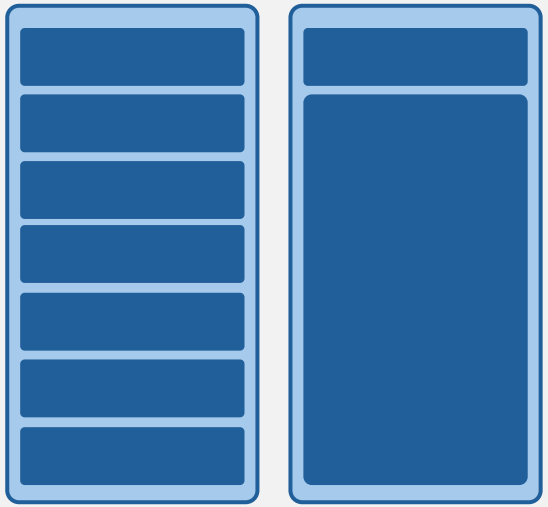


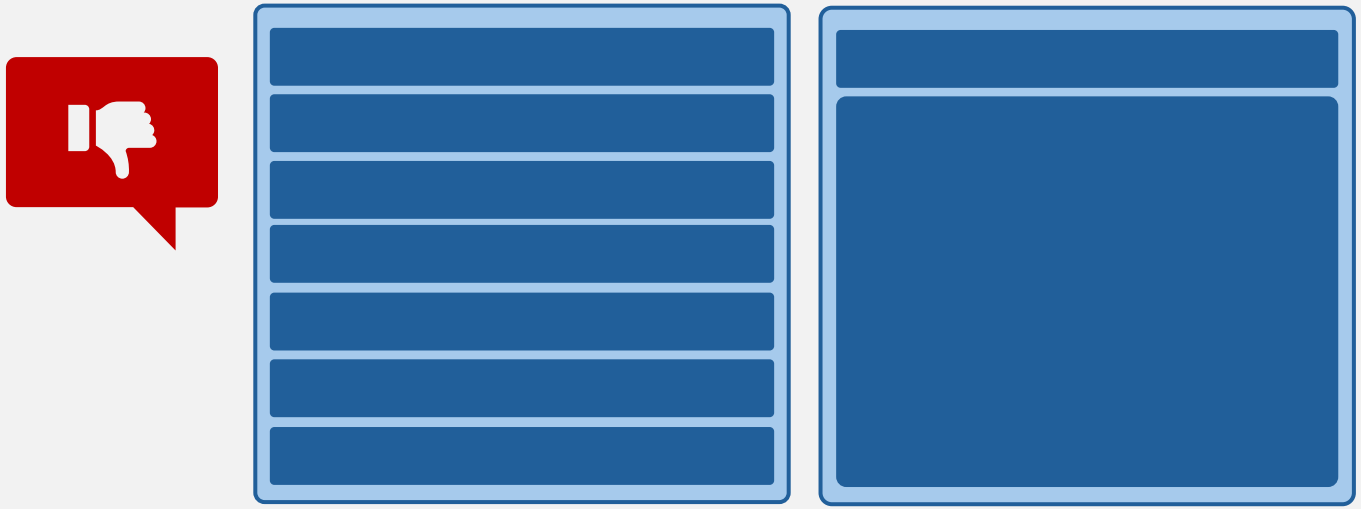
UI ADAPTATIVA

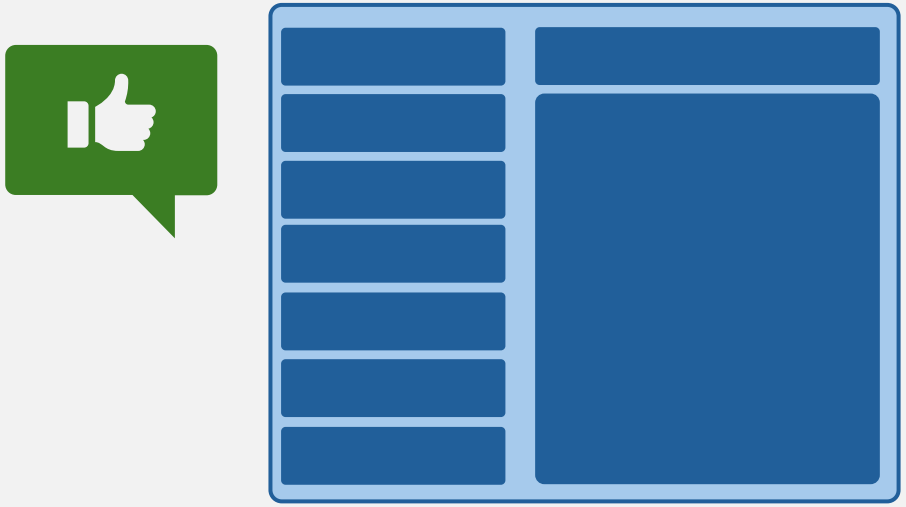
Hasta ahora, los *layouts* en *smartphone* y en escritorio/web, serían los mismos. Es decir, seleccionas un “plato” y te lleva a otra pantalla donde puedes ver la descripción del plato. Sin embargo…

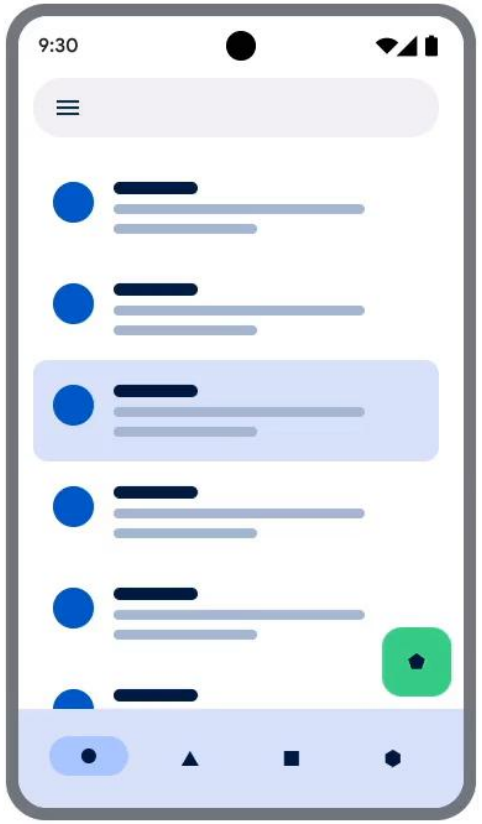
…pensad en el *layout* de Spotify o WhatsApp:

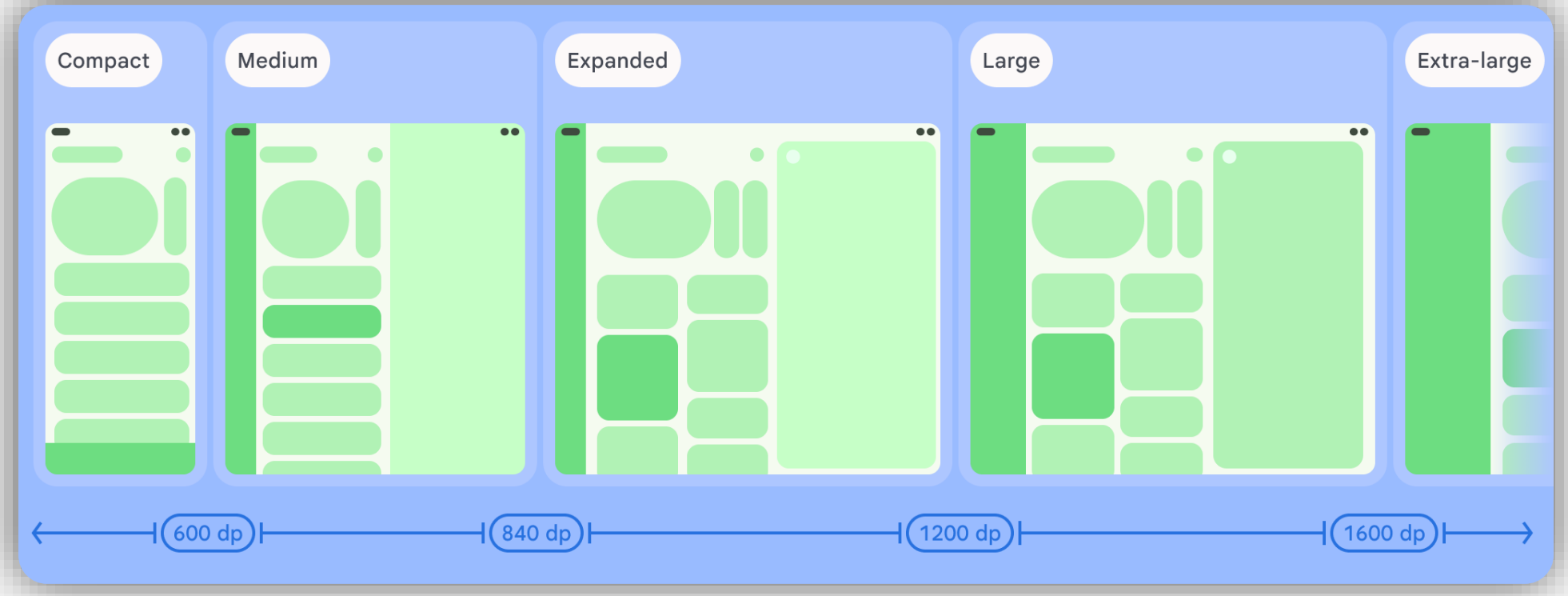
- En smartphone, te cambia entre pantallas al seleccionar y mostrar una canción / lista / chat.
- PERO, en escritorio/web, tienes todo en una sola pantalla.








9:30


Compact
Medium
Expanded
Large
Extra-large
600 dp
840 dp
1200 dp
1600 dp

Diseños canónicos

Detalles de lista



El diseño divide la ventana de la app en dos paneles paralelos: uno para la lista y otro para los detalles. Los usuarios seleccionan elementos de la lista para ver sus detalles. Los vínculos directos en los detalles muestran contenido adicional en el panel de detalles.

El diseño de lista-detalles es ideal para *apps de mensajería*, *administradores de contactos*, o *navegadores de contenido multimedia interactivos*, entre otros.

Detalles de lista



Detalles de lista

```kotlin
@OptIn(ExperimentalMaterial3AdaptiveApi::class)
@Composable
fun MyListDetailPaneScaffold() {
    val navigator = rememberListDetailPaneScaffoldNavigator()
    ListDetailPaneScaffold(
        directive = navigator.scaffoldDirective,
        value = navigator.scaffoldValue,
        listPane = {
            // Listing Pane
        },
        detailPane = {
            // Details Pane
        }
    )
}
```

`navigator`

Gestiona el estado de la navegación. Es el equivalente de **`NavController`**.

`directive`

Informa del tamaño actual de la pantalla y de los posibles paneles.

`value`

Qué paneles están visibles y activos.

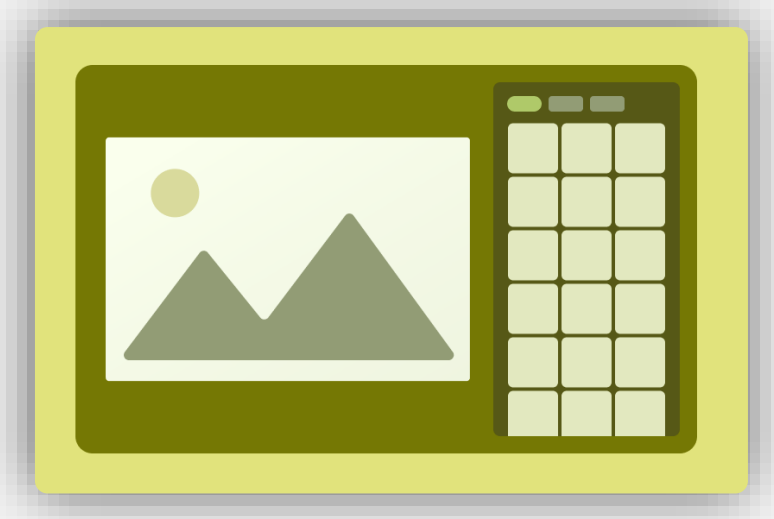
`listPane`

Panel con la lista de elementos.

`detailPane`

Panel con el contenido del elemento seleccionado.

Panel complementario

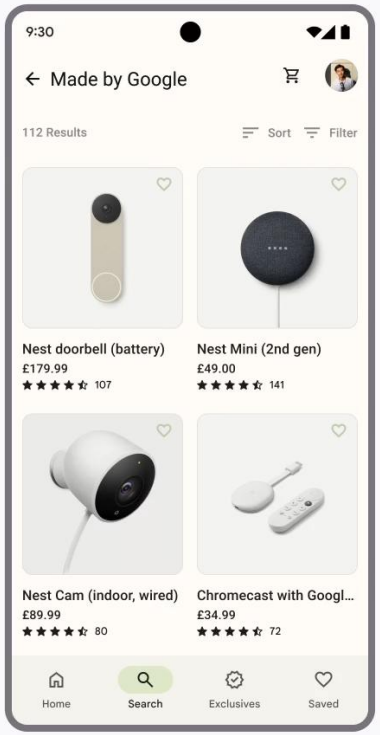


El diseño de panel complementario organiza el contenido de la app en:

1. El área de visualización principal: ocupa la mayor parte de la ventana de la app (por lo general, alrededor de dos tercios) y tiene el contenido principal.
2. El área de visualización secundaria: panel que ocupa el resto de la ventana de la app y presenta contenido complementario del contenido principal.

Son especialmente adecuados para las *apps productividad, música,* o *herramientas y configuración*, entre otros.

Panel complementario



Panel complementario

```kotlin
@OptIn(ExperimentalMaterial3AdaptiveApi::class)
@Composable
fun MySupportingPaneScaffold() {
    // Creates and remembers a navigator to control pane visibility and navigation
    val navigator = rememberSupportingPaneScaffoldNavigator()
    SupportingPaneScaffold(
        // Directive and value help control pane visibility based on screen size and state
        directive = navigator.scaffoldDirective,
        value = navigator.scaffoldValue,
        mainPane = {
            // Main Pane for the primary content
        },
        supportingPane = {
            //Supporting Pane for supplementary content
        }
    )
}
```

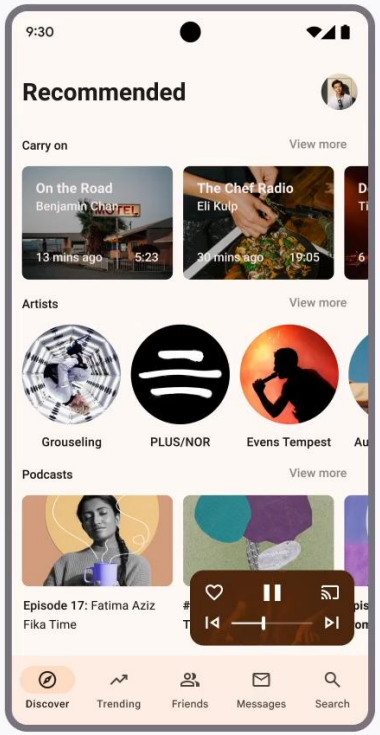
Feed



Un diseño de *feed* dispone elementos en una cuadrícula configurable para permitir una visualización rápida y sencilla de una gran cantidad de contenido.

Los *feeds* son especialmente adecuados para las *apps de noticias y redes sociales*.

Feed


9:30
Recommended
Carry on
View more
On the Road
13 mins ago
5:23
The Chef Radio
Eli Kulp
30 mins ago
19:05
Artists
View more
Grouseling
PLUS/NOR
Evens Tempest
Podcasts
View more
Episode 17: Fatima Aziz
Fika Time
Discover
Trending
Friends
Messages
Search

Feed

```
@Composable
fun MyFeed(names: List<String>) {
    LazyVerticalGrid(
        // GridCells.Adaptive automatically adapts column
        // count based on available width
        columns = GridCells.Adaptive(minSize = 180.dp),
    ) {
        items(names) { name ->
            Text(name)
        }
    }
}
```

A diferencia de los otros, este es un diseño de manera conceptual, es decir, no tiene una clase especial "`FeedPaneScaffold`" o cosas así. En este ejemplo, se propone `LazyVerticalGrid` para crear este diseño.

Diseños canónicos

Acceso a la documentación



```kotlin
@Composable
fun App(appContainer: AppContainer) {
    val navController = rememberNavController()

    NavHost(navController = navController, startDestination = Route.Carta) {

        composable<Route.Carta> {
            val viewModel = remember {
                CartaViewModel(appContainer.obtenerPlatosUseCase)
            }
            CartaScreenRoot(
                viewModel = viewModel,
                onPlatoClick = { plato ->
                    navController.navigate(Route.DetallePlato(plato.id))
                }
            )
        }

        composable<Route.DetallePlato> { backStackEntry ->
            val route: Route.DetallePlato = backStackEntry.toRoute()
            val viewModel = remember {
                DetallePlatoViewModel(
                    platoId = route.platoId,
                    obtenerPlatoPorId = appContainer.obtenerPlatoPorIdUseCase
                )
            }
            DetallePlatoScreenRoot(
                viewModel = viewModel,
                onVolver = { navController.popBackStack() }
            )
        }
    }
}
```

```kotlin
@OptIn(ExperimentalMaterial3AdaptiveApi::class)
@Composable
fun MenuListDetailScreen(
    viewModel: MenuViewModel,
    appContainer: AppContainer
) {
    val navigator = rememberListDetailPaneScaffoldNavigator<String>()
    val scope = rememberCoroutineScope()

    OnBackHandler(enabled = navigator.canNavigateBack()) {
        scope.launch { navigator.navigateBack() }
    }

    ListDetailPaneScaffold(
        directive = navigator.scaffoldDirective,
        value = navigator.scaffoldValue,
        listPane = { /* ... */},
        detailPane = { /* ... */}
    )
}
```

```kotlin
fun App(appContainer: AppContainer) {
    MaterialTheme {

        val cartaViewModel = remember {
            MenuViewModel(appContainer)
        }
        MenuListDetailScreen(
            viewModel = cartaViewModel,
            appContainer = appContainer
        )
    }
}
```

```kotlin
@OptIn(ExperimentalMaterial3AdaptiveApi::class)
@Composable
fun MenuListDetailScreen(
    viewModel: MenuViewModel,
    appContainer: AppContainer
) {
    val navigator = rememberListDetailPaneScaffoldNavigator<String>()
    val scope = rememberCoroutineScope()

    OnBackHandler(enabled = navigator.canNavigateBack()) {
        scope.launch { navigator.navigateBack() }
    }

    ListDetailPaneScaffold(
        directive = navigator.scaffoldDirective,
        value = navigator.scaffoldValue,
        listPane = { /* ... */},
        detailPane = { /* ... */}
    )
}
```

```kotlin
fun App(appContainer: AppContainer) {
    MaterialTheme {

        val cartaViewModel = remember {
            MenuViewModel(appContainer)
        }
        MenuListDetailScreen(
            viewModel = cartaViewModel,
            appContainer = appContainer
        )
    }
}
```

```
listPane = {
    AnimatedPane {
        MenuScreenRoot(
            viewModel = viewModel,
            onDishSelected = { dish ->
                scope.launch {
                    navigator.navigateTo(ListDetailPaneScaffoldRole.Detail, dish.id)
                }
            }
        )
    }
},
```

```
detailPane = {
    AnimatedPane {
        navigator.currentDestination?.contentKey?.let { dishId ->
            val detailViewModel = remember(dishId) {
                DishDescriptionViewModel(
                    appContainer = appContainer,
                    dishId = dishId
                )
            }
            DishDescriptionScreenRoot(
                viewModel = detailViewModel,
                onBackClick = {
                    scope.launch {
                        navigator.navigateBack()
                    }
                }
            )
        }
    }
}
```

```
listPane = {
    AnimatedPane {
        MenuScreenRoot(
            viewModel = viewModel,
            onDishSelected = { dish ->
                scope.launch {
                    navigator.navigateTo(ListDetailPaneScaffoldRole.Detail, dish.id)
                }
            }
        )
    }
},
```

`AnimatedPane` es un componente de *Jetpack Compose* que anima automáticamente la aparición, desaparición y cambio de tamaño de los paneles en diseños adaptativos.

Permite modificar la aparición y desaparición de un panel (**`enterTransition`** y **`exitTrasition`**) por defecto pasando parámetros como **`scaleIn`**, **`fadeIn`**, **`scaleOut`** o **`fadeOut`**.

```kotlin
@OptIn(ExperimentalMaterial3AdaptiveApi::class)
@Composable
fun MenuListDetailScreen(
    viewModel: MenuViewModel,
    appContainer: AppContainer
) {
    val navigator = rememberListDetailPaneScaffoldNavigator<String>()
    val scope = rememberCoroutineScope()

    OnBackHandler(enabled = navigator.canNavigateBack()) {
        scope.launch { navigator.navigateBack() }
    }

    ListDetailPaneScaffold(
        directive = navigator.scaffoldDirective,
        value = navigator.scaffoldValue,
        listPane = { /* ... */},
        detailPane = { /* ... */}
    )
}
```

Para el próximo día

